Diferencia de IMPEDANCIA: dificultad de mapear las estructuras de datos orientadas a objetos (como grafos y jerarquías) a tablas relacionales, debido a las diferencias conceptuales entre el modelo de objetos y el modelo relacional. Caminos directos en objetos se vuelven en consultas JOIN en BDs, o hasta pueden llegar a ser imposibles de hacer.

Persistencia Orientada a Objetos:

Principio de ORTOGONALIDAD: cualquier objeto puede ser persistido, sin importar su tipo. Esto facilita la extensibilidad y mantenimiento del código.

Principio de INDEPENDENCIA: el sistema OO debe saber lo menos posible sobre la tecnología de persistencia subyacente. El código de la aplicación no cambia si se modifica la tecnología de persistencia subyacente.

Persistencia por alcance: establece que todo objeto al cual se puede llegar a partir de un objeto persistente es, a su vez, persistente; es decir, si un objeto persistente conoce a otro, al persistir o modificar el 1°, esa acción se propaga automáticamente. Reduce el n° de saves (ideal para independencia), pero dificulta saber dónde terminar la propagación de la persistencia -> pues no queda claro cuál es el "límite" del grafo de objetos.

MAPEO DE JERARQUIAS:

- Tabla por Jerarquía (*Single Table*): toda la jerarquía se mapea a 1 sola tabla, con 1 columna que indica el tipo.

Ventaja: consultas rápidas sin JOINs (es polimorfismo friendly).

Desventaja: campos nulos.

- Tabla por Clase (*Joined*): cada clase tiene su tabla, incluyendo las abstractas, que guarda sólo sus atributos.

Ventaja: mantenimiento y ahorro de espacio (evita nulls). La + adecuada desde el punto de vista de la normalización. Útil para el caso en que la clase padre no sea abstracta (sea concreta).

Desventaja: los JOINs que tengo que hacer para recuperar 1 objeto.

- Tabla por Clase Concreta (*Table per Class*): solo se mapean las clases concretas, replicando los atributos del padre en c/u.

Ventaja: simple, no requiere JOINs p/ obtener 1 instancia completa, un mejor aprovechamiento del espacio (en desuso porque el espacio es barato).

Desventaja: columnas duplicadas y UNIONs para consultas polimórficas.

El atributo `discriminator-value` se usa sólo en el mapeo de herencia Single-Table para indicar qué valor concreto debe almacenarse en la columna discriminadora cuando se persiste una instancia de esa clase en la BD. Al leer filas de la tabla, Hibernate mira el valor de esa columna y, según el discriminator-value, decide qué subclase instanciar.

Operaciones en cascada: `Cascade` establece la persistencia por alcance (indica si las operac realizadas sobre 1 entidad deben propagarse a sus entidades relacionadas). Por defecto nada se propaga en JPA. Puede ser ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH.

El tipo de `Fetch` puede ser EAGER (carga la relación automáticamente junto con la entidad principal, aunque no se necesite, de modo que al obtener el objeto ya incluye todos los elementos) o LAZY (utiliza el patrón proxy -> carga la relación sólo cuando se accede explícitamente, lo que mejora la performance si no siempre se usa, es lo default).

CONCURRENCIA -> 2 tipos de bloqueo:

- pesimista: previene el conflicto bloqueando accesos (no permite que nadie pueda modificar lo que estoy modificando yo).

- optimista: permite el conflicto y lo detecta al momento de la escritura, usando versionamiento (asume q la situación de q alguien cambie lo q yo estoy cambiando no sucederá, así q lo permite).

Versionamiento: n° de versión o timestamps, para detectar conflictos de actualización. Si la versión del objeto en la base de datos es diferente a la que el usuario leyó, la transacción falla.

Control de concurrencia optimista basado en versiones que utiliza Hibernate: se observa el @Version al escribir: si el n° es el mismo de la base, ok. Si no (o sea, tengo un n° distinto al que se leyó), considera que tengo una versión vieja y lanza una excepción.

Caching: es una manera de mantener los diferentes estados de los objetos sincronizados. La idea es minimizar la cantidad de acceso a la BD -> son lentos.

Caché en Hibernate: soporta caching de Nivel 1 (por sesión, activado por defecto) y Nivel 2 (a nivel de SessionFactory, no implementado por defecto pero hay productos que lo hacen).

Patrón DAO: establece que por c/ obj de dominio persistente debe existir 1 clase que persiste y recupera sus instancias. Es decir, un DAO se centra en operaciones de CRUD (create, read, update, delete) para 1 entidad específica. Se implementan con interfaces. Problemas:

- Granularidad / relaciones e/ DAOs de clases relacionadas.
- "Mudanza" del comportam de negocio a la capa de persistencia / el modelo queda anémico (sólo getters y setters).
- Acoplamiento con la tecnología de persistencia subyacente.
- Si se coloca 1 ORM directam dentro del DAO, se desaprovechan las capacid de dicho ORM.

Patrón Repository: ya no representa la capa de persistencia, sino la colección de objetos persistentes. Se enfoca en el "fetch" -> aprovecha los updates del ORM. Ventaja: al abstraer la colección de obj, contribuye a que el sist OO se preocupe lo - posible por la tecnología de persistencia subyacente, alineándose con el Principio de Independencia en los ORMs.

En resumen, la diferencia principal es que el DAO se enfoca en la persistencia de instancias individuales de objetos de dominio y a menudo está + acoplado a la tecnología de persistencia, mientras que el Repository se centra en la colección de objetos y proporciona una interfaz + abstracta para acceder y gestionar esos objetos, desacoplando el dominio de los detalles de la persistencia subyacente.

Modelo Relacional -> Pros: SQL (única tecnología con un estándar real) - súper simple (juego de muy poquitas piezas) - integridad y consistencia (transac ACID). Contras: dif de impedancia.

BDs NO-SQL preparadas para escalar fácilmente.

Lo que permite que el balance de carga sea posible (en NO-SQL) es la distribución horizontal de datos (sharding). Y estas funcionan muy bien c/ la agregación -> es - probable tener q hacer joins.

Característica	Docum embebidos o array de IDs	Colección separada con referenc
Rendimiento	Mejor p/ consultas conjuntas	Mejor p/ relaciones grandes
Escalabilidad	Limitado por 16 MB x documento	Escala mejor a millones
Normalización	Baja (más duplicación)	Alta (menos duplicación)
Mantenimiento	Más simple de leer y escribir	+ complej (requier joins manuales)

Escalabilidad en MongoDB (y NoSQL en general):

Replicación (*Replica Sets*): conjunto de nodos que comparten la misma info p/ alta disponibilidad y redundancia. Si el nodo primario cae, 1 de los secundarios toma su lugar.

Sharding: permite la partición de los datos a nivel de colecciones, distribuyéndolos e/ dist nodos (shards) para escalar el almacenamiento y la carga. La "shard key" es un campo (o campos) inmutable que determina cómo se particionan los datos. Los "chunks" son rangos contiguos de datos de una colección shardeada que se distribuyen entre los shards. La elección de una buena shard key es importante (y difícil) porque determina la búsqueda de las queries.

Una buena shard key es aquella que tiene muchos valores diferentes posibles, distribuye de forma pareja y no crece de forma monotónica; todo p/ garantizar una distribuc equilibrada de los datos e/ los shards y evitar una sobrecarga en 1 sólo shard. Por ej, los últimos dígitos de 1 DNI o 1 n° de teléfono, NO 1 fecha de nacimiento o 1 ciudad.

Teorema CAP: establece que en un sistema distribuido, solo se pueden garantizar 2 de estas 3 propiedades al mismo tiempo:

- Consistencia (C): todas las réplicas tienen la misma versión de los datos y ninguna lectura obtiene un dato obsoleto.

- Disponibilidad (A): el sistema permanece activo y todos los clientes pueden leer y escribir, incluso con fallos de nodos.

- Tolerancia a Particiones (P): el sistema sigue funcionando a pesar de fallos en la red que dividen el cluster en particiones que no pueden comunicarse.

Las RDBMS suelen ser CA (Consistencia + Disponibilidad, sin tolerancia a particiones), mientras que las NoSQL priorizan AP (Disponibilidad + Tolerancia a Particiones) o CP (Consistencia + Tolerancia a Particiones).

- Disponibilidad + Tolerancia a particiones: si se corta la conexión entre 2 nodos, sigo teniendo disponibilidad (y consistencia LOCAL), pero sacrifico la consistencia.

- Consistencia + Tolerancia a particiones: si espero a que vuelvan a estar conectados los nodos.

¿Cuántos nodos necesitaría para asegurarme consistencia? a todos los nodos. Si tengo todos los nodos con el dato más nuevo, siempre voy a leer el dato actualizado. Se suele aceptar cuando la mayoría está actualizada -> w > n/2. Por esto los nodos tienen que ser IMPARES.

CASSANDRA: tiene almacenamiento por columnas. No tiene JOINs -> promueve un "Query-First Approach" -> el modelado de datos se basa en las consultas que se van a realizar, priorizando la velocidad de lectura sobre la normalización -> toda consulta de diseña para ser buscada en 1 sola tabla. Por ej, el promedio de las edades es + rápido que en almacenam x filas porque sólo tengo que leer 1 columna.

Concepto clave: el *consistency level* (CL) determina cuántas réplicas deben confirmar una operación para que ésta se considere exitosa — el *quorum* es la práctica de requerir respuestas de una mayoría o conjunto definido de réplicas.

Fórmula útil: si `RF` = factor de replicación, para garantizar consistencia fuerte conviene que `R + W > RF` (R = réplicas leídas, W = réplicas escritas). Ej.: `RF = 3` → `W = 2` y `R = 2` cumple `2 + 2 > 3`.

Data Warehouse: sistema diseñado para almacenar grandes volúmenes de datos de múltiples fuentes para análisis y generación de informes, no para transacciones diarias. **Características**:

- Integración y Consistencia: unifica datos de diversas fuentes.
- Variable en el Tiempo y No Volátil: los datos se mantienen a lo largo del tiempo y no se modifican ni se eliminan.
- Organización Temática: los datos se organizan por temas de negocio.

Funciones y Uso: Business Intelligence (BI), informes gerenciales, análisis de tendencias, Data Mining y Machine Learning.

Componentes: Fuentes de datos, herramientas ETL (Extracción, Transformación, Carga), metadata, base de datos del DW y herramientas de consulta/análisis.

Data Mart: subconjunto del Data Warehouse, orientado a un área específica del negocio (e.g., ventas, finanzas) para análisis detallado.

Data Lake: repositorio para almacenar grandes cantidades de datos, tanto estructurados como sin estructurar, sin necesidad de transformarlos de inmediato. Permite diversos tipos de análisis.

Tablas:

- De hechos: contienen medidas sobre un proceso de negocio. Tabla central del análisis. Muy grandes y desnormalizadas o en 3FN.

- De dimensiones: factores por los que se analiza el negocio (tiempo, producto, cliente). Contienen metadatos sobre los hechos. Más chicas y desnormalizadas. EJ muy común: fechas.

3 modelos:

- Estrella: modelo principal y el más simple, 1 tabla de hechos en el centro y alrededor tablas de dimesiones. Ventaja: para obtener información, a lo sumo tengo 1 JOIN. Desventaja: consume más almacenamiento, carga de datos más lenta al trabajar con datos desnormalizadas.

- Copo de nieve: evolución del estrella. Ventaja: tablas de dimesiones están normalizadas -> permite más contexto, evitando la redundancia. Desventaja: implica + JOINs en las consultas.

- Galaxia o Constelación: múltiples tablas de hechos que comparten tablas de dimensiones.

Hibernate: es una herramienta de mapeo objeto/relacional (ORM) para ambientes Java. Framework de persistencia, basado en objetos, para BBDDs relacionales. Surge por la necesidad de persistir objetos Java en BBDDs relacionales. Su problema es que es 1 herram, no 1 estándar.

JPA (Java Persist API): es un estándar de persistencia p/ Java que permite mapear objetos Java a 1 BD relacional. Basado en Hibernate. Introduce el concepto de Entities y la Convention over Configur.., donde x defecto se persisten todos los atributos a - que se marquen como @Transient.

Fue creada para unificar y simplificar la forma de persistir objetos Java en BDs relacionales, definiendo 1 conj de anotaciones y 1 API común sobre la que pueden apoyarse distintos proveedores (Hibernate, EclipseLink, etc). Con JPA se evita depender de APIs propietarias de cada framework y se favorece la portabilidad de la capa de persistencia.

DTOs (Data Transfer Objects): son objetos simples que se utilizan p/ transferir datos e/ capas, especialmente para sacar info de 1 contexto transaccional y poder trabajar con ella fuera de la transac sin mantenerla abierta. No contienen lógica de negocio, sólo atributos, getters y setters.

Se usan especialmente cuando hay fronteras claras e/ módulos o aplicaciones, p/ reducir el n° de llamadas e/ capas (por ej, en arquitecturas en capas o microservicios).

Reglas para persistencia de POJOs HIBERNATE: hay 4 reglas principales a seguir:

- implementar un constructor sin argumentos.
- proveer un identificador único (@Id).
- la clase no tiene que ser final (p/ poder usar proxies p/ asociaciones lazy).
- declarar setters y getters para los campos persistentes (acceso p/ las variables).